Forecasting Daily Mean Temperature at Vancouver International Airport Using Time-Series Methods

Jiahao Li Da Yang Haoyu You Yuhao Zhang

STAT 443 Team Project

Abstract

In this project, we ask whether lagged precipitation variables can help improve one-step-ahead forecasts of daily mean temperature at Vancouver International Airport. We use daily weather data from Environment and Climate Change Canada from 2016-01-01 to 2025-12-31, and compare simple benchmark rules, exponential smoothing, ARIMA models, and ARIMAX models with lagged weather variables and Fourier seasonal terms. We evaluate the models using recursive one-step forecasts on the 2024-01-01 to 2025-12-31 holdout period, with RMSE as the main measure. Under our final screening-and-update workflow, the best model is ARIMA(2,0,2), with holdout RMSE about 1.600. The best ARIMAX finalist, ARIMAX(2,0,0), is very close at about 1.604, and both clearly beat ETS at about 1.683. Overall, the results suggest that most of the short-run forecasting signal is already coming from seasonality and autocorrelation in the temperature series, while lagged precipitation variables only help a little.

1 Data and Variables

The data come from the Environment and Climate Change Canada Historical Climate Data portal for station `VANCOUVER INTL A` (Climate ID 1108395). The full dataset runs from 2016-01-01 to 2025-12-31 and has 3,653 daily observations. Our response variable is daily mean temperature in degrees Celsius. The main explanatory variables we look at are lagged total precipitation, lagged rain, and lagged snow. To deal with seasonality, we also include Fourier terms based on day of year.

The summary statistics show that the average daily temperature is about 10.66°C, with values ranging from -11.6°C to 26.4°C. Missingness in the main variables is fairly small: the response variable has 65 missing values (about 1.8%), and the precipitation-related variables each have less than 1% missingness. Total precipitation is very right-skewed, with a median of 0 and a few large rain events, while snowfall is 0 on most days but jumps up on some winter days.

The time plot in Figure 1 shows a very clear yearly pattern, with colder winters and warmer summers repeating over time. The autocorrelation plot in Figure 2 also shows that the series is highly persistent, which is one reason why autoregressive models and persistence forecasts make sense here. Since the seasonal pattern is smooth and repeats from year to year, Fourier terms are a simple way to capture that pattern.

1.1 Summary Statistics

Table 1: Summary statistics for the main variables used in the project, including sample size, missing values, and the main distribution summaries.

variable	n	missing	mean	sd	min	median	max
mean temp c	3653	65	10.655	6.026	-11.600	10.300	26.400
max temp c	3653	63	14.191	6.626	-7.800	13.600	32.400
min temp c	3653	59	7.072	5.716	-15.300	7.200	20.400
total precip mm	3653	19	3.139	6.437	0	0	67.100
total rain mm	3653	30	3.039	6.370	0	0	67.100
total snow cm	3653	21	0.107	1.043	0	0	27.800

1.2 Time Plot

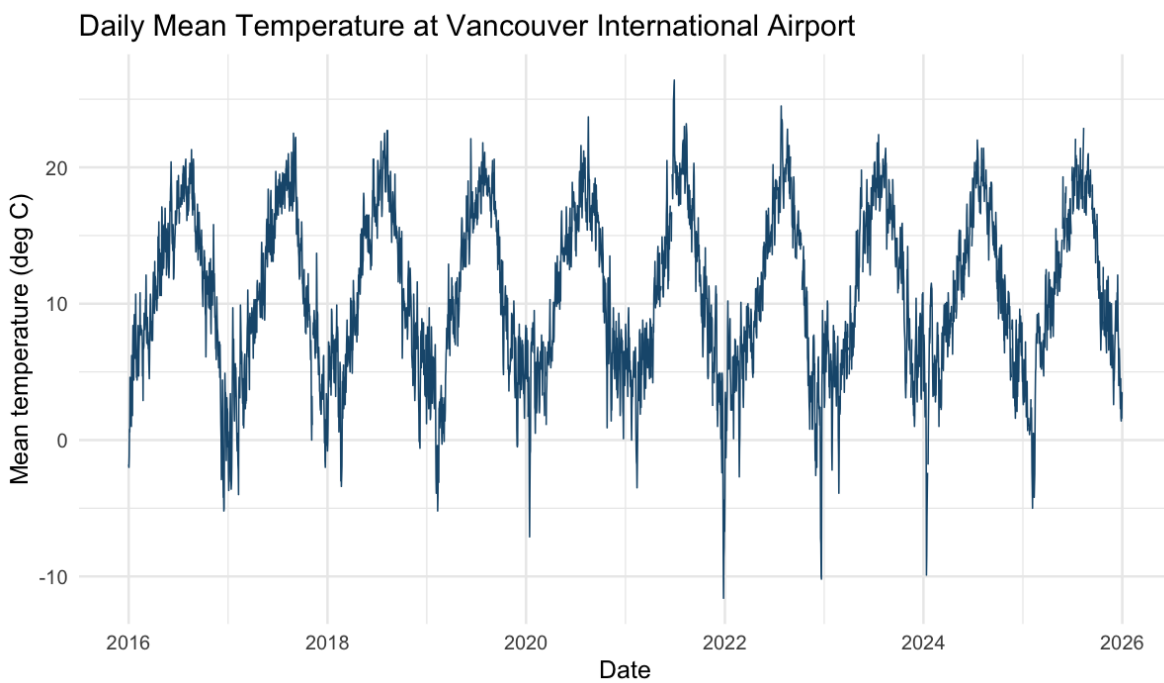

Figure 1: Daily mean temperature at Vancouver International Airport from 2016-01-01 to 2025-12-31. The plot shows the strong repeating yearly pattern, with lower winter temperatures and higher summer temperatures.

1.3 ACF Plot

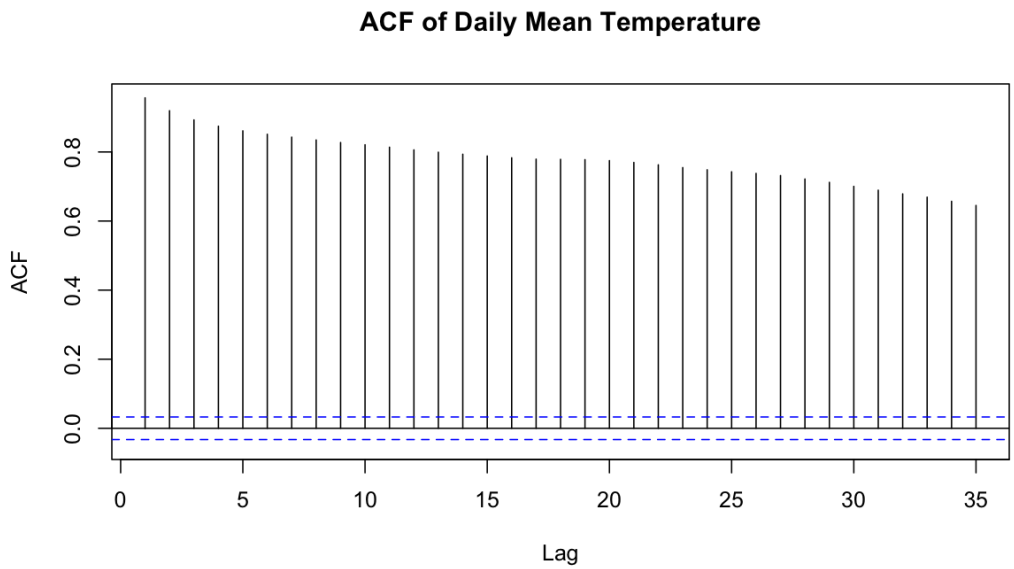

Figure 2: Autocorrelation function for the daily mean temperature series. The large early-lag correlations show strong persistence, which helps explain why autoregressive-style models are reasonable for this dataset.

2 Methods

We check forecast performance using recursive one-step-ahead forecasts on a holdout period that comes after the training period. The training period runs from 2016-01-01 to 2023-12-31, and the holdout period runs from 2024-01-01 to 2025-12-31. Because we use lagged variables and only keep dates where the needed weather inputs are available, the final holdout comparison is based on 687 usable forecast dates. For each holdout date, the model only uses information available up to the day before and then makes a one-step forecast. Our main comparison measure is holdout RMSE, and we also report in-sample 1-step RMS prediction errors for the main methods.

We compare four groups of methods:

1. Simple benchmark rules: persistence and the training-set mean.
2. Exponential smoothing using an ETS model.
3. ARIMA models with Fourier seasonal terms.
4. ARIMAX models with lagged precipitation, lagged rain, lagged snow, and Fourier seasonal terms.

The persistence benchmark uses $\hat{y}_t = y_{t-1}$, while the training-mean benchmark just uses the average temperature from the training data. The ETS model is chosen using the usual exponential smoothing tools. For ARIMA and ARIMAX, we use `auto.arima()` as a starting point, but we also check nearby (p, d, q) orders instead of just taking the automatic choice and stopping there. In the ARIMA models, Fourier terms are used to handle seasonality. In the ARIMAX models, we include

lagged precipitation variables along with the same Fourier terms. We choose the final models using holdout RMSE instead of only AIC, because the model that fits the training data best is not always the one that forecasts best later on.

More specifically, we did not try to search over every possible order. We used the automatic model as a starting point, built a small set of nearby candidates by changing p and q up or down, and then used a two-stage screening rule. In the first stage, we ranked the nearby candidates by training-period AIC, which is quick to compute. In the second stage, we carried only the top screened finalist from each family into the recursive one-step holdout comparison. We kept the final set small because full holdout one-step evaluation takes a lot more time than the screening step, and we wanted the final comparison to stay realistic for a class project while still checking that we were not just accepting one automatic fit without question. For those finalists, we estimated the model on the training period, kept the chosen model structure fixed, and then updated the fitted state through the holdout period instead of fully re-estimating parameters every day. This stays pretty close to the course case-study setup based on training-set estimates, but keeps the computation manageable.

We also checked ARIMA diagnostics on the training period, mainly by looking at residual autocorrelation and Ljung–Box results for the candidate models. We did not use that as the only decision rule, but it helped us rule out weaker ARIMA choices before focusing on the holdout RMSE comparison.

3 Results

3.1 Main RMSE Comparison

Table 2: Holdout RMSE comparison for all competing forecasting methods. Lower RMSE means better one-step forecast performance on the 2024–2025 holdout period.

model	holdout rmse
ARIMA (2,0,2)	1.600
ARIMAX (2,0,0)	1.604
ETS	1.683
Persistence	1.684
Training mean	5.737

Table 3: In-sample 1-step RMS prediction error comparison for the benchmark methods and the best ARIMA and ARIMAX models. Lower values mean better one-step prediction performance on the training period.

model	n used	in sample rmspe
Persistence	2866	1.766
Training mean	2892	6.090
ETS(A,N,N)	2893	1.767
ARIMA (2,0,2)	2893	1.656
ARIMAX (2,0,0)	2852	1.651

Table 2 shows that the top screened ARIMA and ARIMAX finalists both do much better than the benchmark methods. The best overall model is ARIMA(2,0,2), with holdout RMSE 1.600. The top ARIMAX finalist, ARIMAX(2,0,0), is very close at 1.604, so the gap is small but still slightly in favor of the univariate ARIMA model under our final workflow.

Among the benchmark and simpler methods, ETS is the strongest competitor with RMSE 1.683, just ahead of persistence at 1.684. The training-mean rule does very badly, with RMSE 5.737, which makes sense because this series clearly has seasonality and short-run dependence. The automatic exponential smoothing fit selected ETS(A,N,N), so the best ETS benchmark turned out to be a simple level-only smoothing model without trend or seasonality terms built directly into the ETS form. A short summary of that ETS choice is included in Table 4 in the appendix.

The in-sample 1-step RMS prediction errors tell a pretty similar story. ARIMA(2,0,2) and ARIMAX(2,0,0) are again the two strongest models, while the simple benchmark rules are clearly worse. In the training period, ARIMAX(2,0,0) is slightly lower than ARIMA(2,0,2), but that small edge does not carry over to the holdout comparison.

The candidate-model comparison is still useful because it shows how the finalists were chosen. We first screened nearby candidates using training-period AIC, and then carried only the top ARIMA and top ARIMAX models into the recursive holdout comparison. So even though this was not a full exhaustive search, the final ranking still comes from a clear two-stage process rather than just blindly trusting one automatic fit.

The ARIMA diagnostics tell a similar story. Table 6 in the appendix lets us compare information criteria, Ljung–Box results, and holdout RMSE side by side. The screened winner, ARIMA(2,0,2), also looks reasonable from the residual checks, so the final choice is backed up by both diagnostics and holdout performance.

3.2 Best Holdout Forecasts

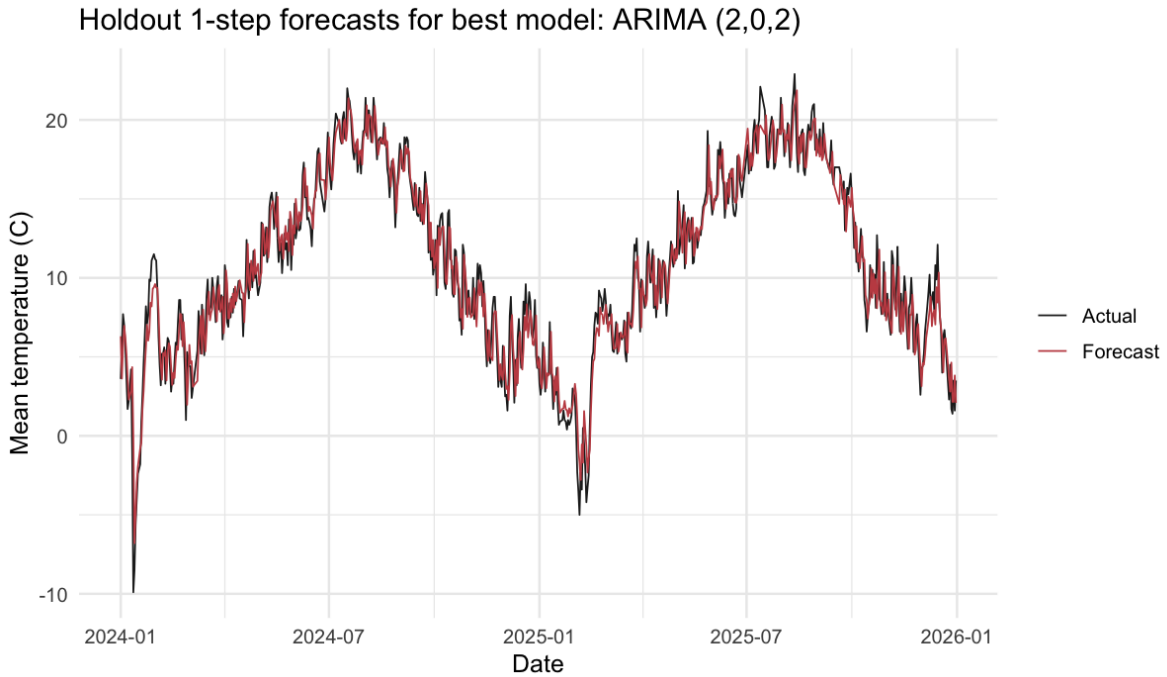

Figure 3: Actual values and one-step holdout forecasts from the best-performing ARIMA(2,0,2) model over the 2024–2025 holdout period. The figure shows that the model follows the overall seasonal pattern well and stays fairly close to the observed series for most dates.

Figure 3 shows that the best model follows the overall seasonal pattern pretty closely during the holdout period and also reacts reasonably well to shorter-run changes. The margin over the top ARIMAX finalist is small, which tells us the two best approaches are fairly close in practice. For the selected ARIMA(2,0,2) model, the mean holdout forecast error is close to zero and the residual plots do not suggest major leftover structure.

4 Discussion

The biggest pattern in this series is strong seasonality along with short-run persistence. That helps explain why both ETS and ARIMA-style models do much better than the training-mean rule. It also helps explain why the best univariate ARIMA model already performs very well even before adding external regressors.

In our final comparison, the ARIMAX finalist does not quite beat the top ARIMA finalist. The difference is small, but it suggests that lagged precipitation variables do not add much forecasting power beyond what is already captured by the temperature series and the seasonal terms. So the answer to our project question ended up being a bit more modest than we first expected: explanatory variables are still reasonable to try, but in this dataset they do not clearly beat a strong univariate ARIMA model.

Another thing we learned is that `auto.arima()` is useful as a starting point, but it should not be the final decision by itself. Comparing nearby candidate models using holdout RMSE makes more sense here, since this project is mainly about forecast performance, not just in-sample fit.

We also think the candidate search was enough for the scope of this project. Once we narrowed the field with a quick screening step and then compared the top finalists carefully, the overall picture became pretty stable. If we had more time, we could check a wider set of candidates, but for this class project the current comparison gives a clear answer.

5 Conclusion

Our results show that a strong univariate ARIMA model gives the best one-step-ahead daily temperature forecasts at Vancouver International Airport under our final workflow. The top ARIMAX finalist is still very competitive and clearly beats the benchmark methods, but it does not improve on the best ARIMA model. Overall, this suggests that seasonality and autocorrelation are doing most of the forecasting work in this series, while lagged external weather variables only add a small amount of extra help.

6 Contributions

Team formation. Our team came together because we all liked the idea of doing a weather forecasting project with real daily data. We looked through several possible datasets and talked about which one would actually be realistic for the course requirements, and in the end we agreed that the Vancouver daily weather data gave us the best balance between a clear forecasting question and data we could realistically clean and model within the term.

Author order. The author order is alphabetical by surname.

Member contributions.

Da Yang: took part seriously in evaluating and choosing the dataset, planned the proposal workflow and task split, contributed to the proposal work and review, took the lead in organizing the overall final project workflow and the division of tasks, contributed the largest share of the code implementation and revisions, joined the final code review, wrote and revised the final PDF report, and kept the group moving by following up on code review and project progress.

Yuhao Zhang: found the full raw dataset that the group eventually chose for the project, took part in evaluating and selecting the dataset, contributed to the proposal task split and review, gave useful suggestions for improving the final project, contributed a large amount of code implementation and revisions, joined the final project code review, and helped write and revise the final PDF report.

Haoyu You: was very involved in evaluating candidate datasets, helped explain some of the project ideas and forecasting concepts to the group early on when the overall direction was still not very clear, took part in the proposal task split and review, gave helpful suggestions for improving the final project, contributed a large amount of code implementation and revisions, joined the final project code review, and helped write and revise the final PDF report.

Jiahao Li: found several datasets that were seriously considered in the early stage even though they were not selected in the end, took part seriously in evaluating and selecting the final dataset, contributed to the proposal task split and review, suggested useful project improvements, contributed a large amount of code implementation and revisions, joined the final project code review, and helped write and revise the final PDF report.

AI use statement. We used ChatGPT to help with project planning, thinking through the overall workflow, getting suggestions for task division, and supporting some of the code review. The shared ChatGPT conversation used for this support is available at `https://chatgpt.com/share/69d4dde6-17c4-83e8-aa6c-9c2965c7ed1d`.

A Additional Tables and Figures

The first table gives a short summary of the ETS form chosen on the training period.

Table 4: Summary of the ETS model chosen on the training period. This shows the automatic exponential smoothing form used in the final comparison.

method	error type	trend type	season type	damped
ETS(A,N,N)	A	N	N	FALSE

The table below lists the candidate ARIMA and ARIMAX models that were checked after using the automatic model as a starting point. The table also shows the quick screening step, so it is clear which nearby models were ranked highly and which single finalist from each family was carried into the recursive holdout comparison.

Table 5: Candidate ARIMA and ARIMAX models from the quick screening step and the final rolling holdout comparison. The screening rank is based on training-period AIC, and the finalist column shows which single model from each family was carried forward to the full rolling evaluation.

model label	family	p	d	q	screening aic	screening rank	finalist	holdout rmse
ARIMA (1,0,1)	ARIMA	1	0	1	11154.670	2	FALSE	NA
ARIMA (1,0,2)	ARIMA	1	0	2	11156.560	4	FALSE	NA
ARIMA (2,0,1)	ARIMA	2	0	1	11156.600	5	FALSE	NA
ARIMA (2,0,2)	ARIMA	2	0	2	11153.950	1	TRUE	1.600
ARIMA (2,0,3)	ARIMA	2	0	3	11155.290	3	FALSE	NA
ARIMA (3,0,2)	ARIMA	3	0	2	11160.320	6	FALSE	NA
ARIMA (3,0,3)	ARIMA	3	0	3	11162.630	7	FALSE	NA
ARIMAX (1,0,0)	ARIMAX	1	0	0	10986.650	5	FALSE	NA
ARIMAX (2,0,0)	ARIMAX	2	0	0	10980.830	1	TRUE	1.604
ARIMAX (2,0,1)	ARIMAX	2	0	1	10982.820	3	FALSE	NA
ARIMAX (3,0,0)	ARIMAX	3	0	0	10982.820	2	FALSE	NA
ARIMAX (3,0,1)	ARIMAX	3	0	1	10984.650	4	FALSE	NA

The next table adds a simple diagnostic check for the ARIMA candidates. Here we report AIC, BIC, Ljung–Box p-values, and holdout RMSE, so the reader can see both the residual diagnostics and the final forecast comparison in one place.

Table 6: Diagnostic summary for the ARIMA candidates fit on the training period, including information criteria, Ljung–Box p-values, and holdout RMSE. This helps compare residual diagnostics with forecasting performance.

model	aic	bic	ljung box p value	holdout rmse
ARIMA (2,0,2)	11153.950	11225.590	0.688	1.600
ARIMA (1,0,1)	11154.670	11214.370	0.510	NA
ARIMA (2,0,3)	11155.290	11232.910	0.673	NA
ARIMA (1,0,2)	11156.560	11222.230	0.447	NA
ARIMA (2,0,1)	11156.600	11222.270	0.445	NA
ARIMA (3,0,2)	11160.320	11237.930	0.288	NA
ARIMA (3,0,3)	11162.630	11246.210	0.245	NA

The next table gives the same kind of quick diagnostic summary for the best ARIMAX model.

Table 7: Diagnostic summary for the best ARIMAX model fit on the training period, including information criteria, a Ljung–Box p-value, and holdout RMSE.

model	aic	bic	ljung box p value	holdout rmse
ARIMAX (2,0,0)	10980.830	11064.210	0.479	1.604

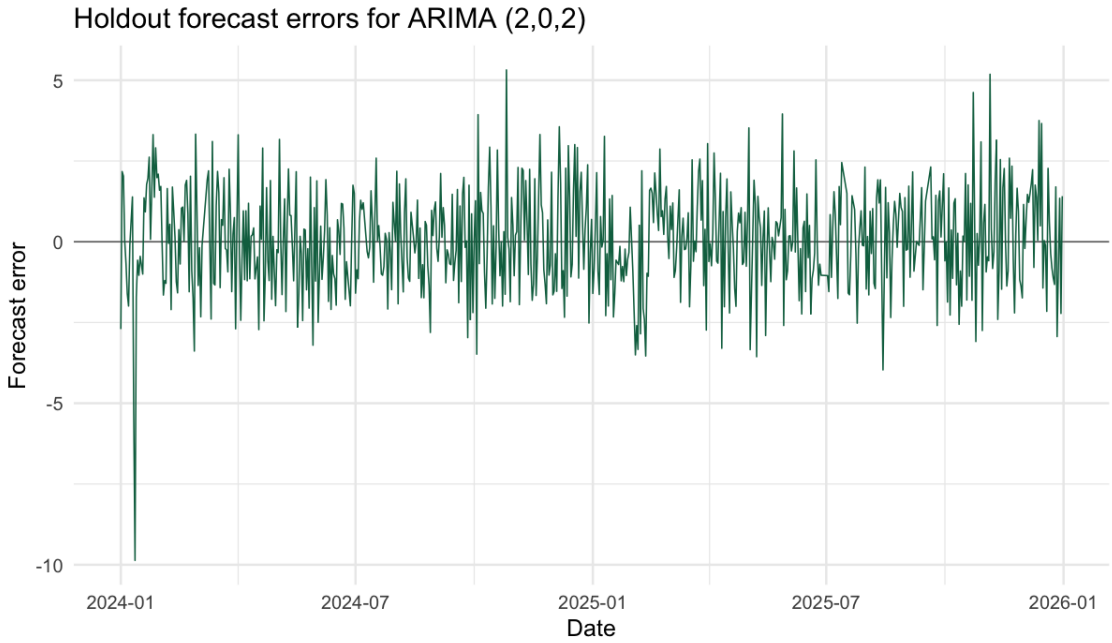

Figure 4: Holdout forecast errors over time for the best-performing model. The errors stay centered around zero overall, although there are a few larger misses during more extreme periods.

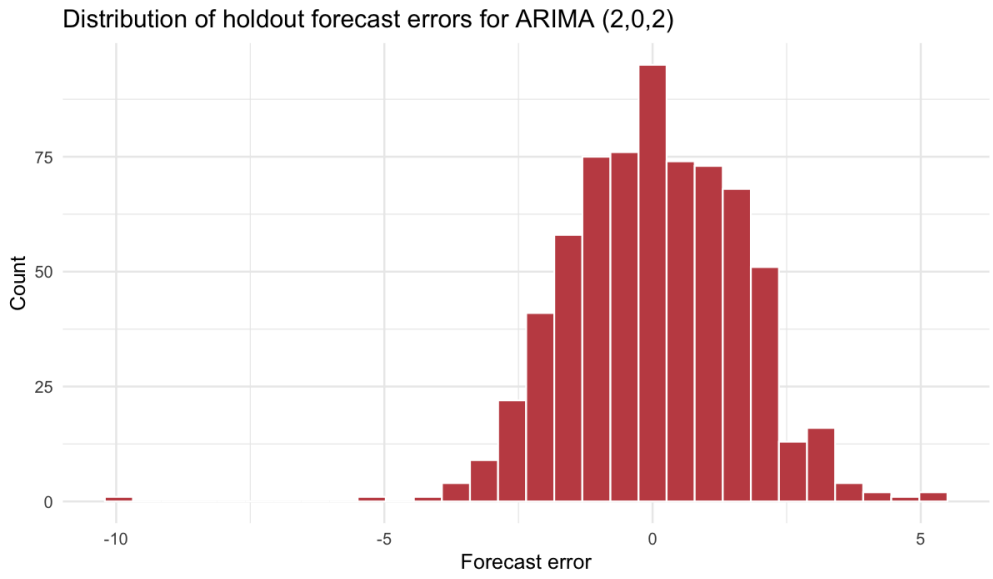

Figure 5: Distribution of holdout forecast errors for the best-performing model. Most errors are concentrated near zero, with a few larger positive and negative values in the tails.

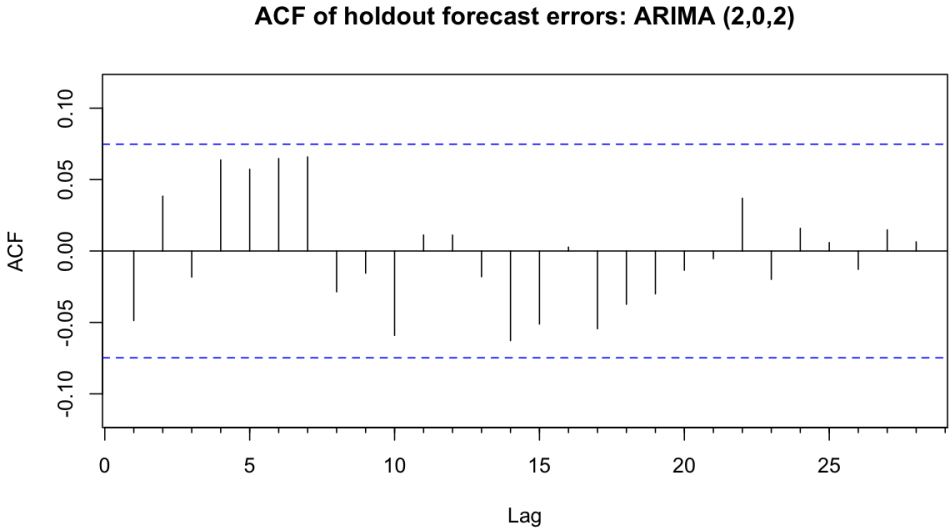

Figure 6: Autocorrelation of holdout forecast errors for the best-performing model. This plot is used to check whether there is still obvious leftover serial dependence after forecasting.

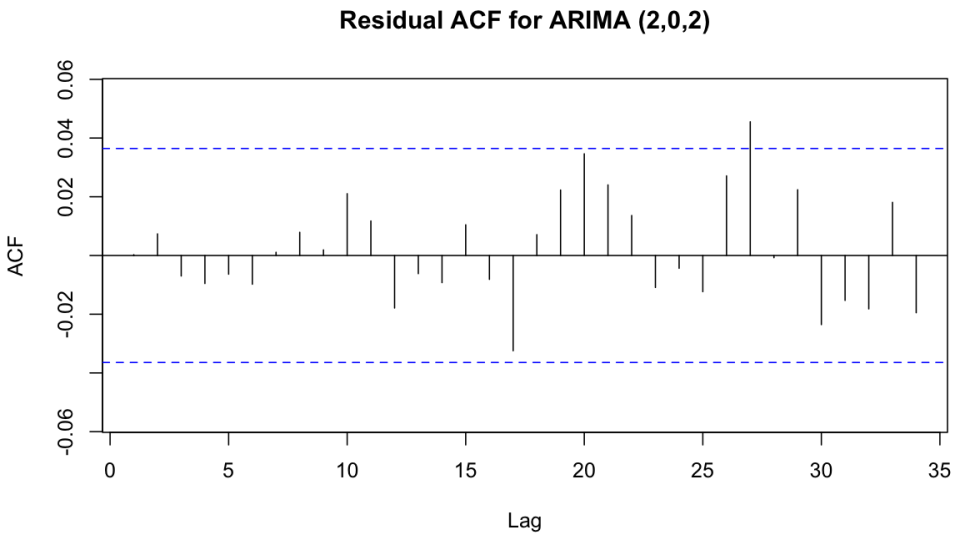

Figure 7: Residual ACF for the best ARIMA candidate fit on the training period. This helps show whether the chosen ARIMA model leaves strong residual autocorrelation before we move to the holdout comparison.

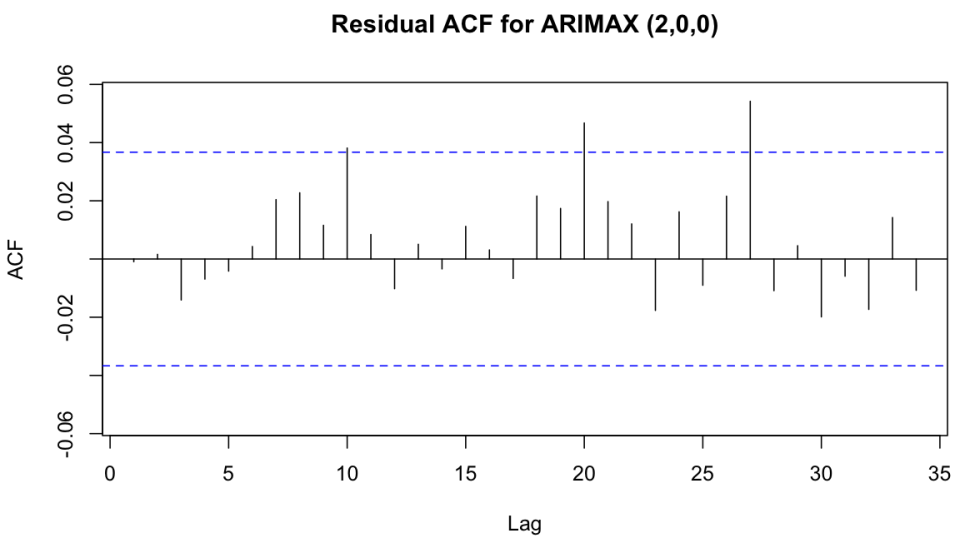

Figure 8: Residual ACF for the best ARIMAX candidate fit on the training period. This gives a similar residual check for the best model that uses lagged external weather variables.

B Selected R Code

This appendix includes a few of the main R code blocks used in the project. The full `R` and `Rmd` files are also included in the submitted archive, but the listings below show the main parts of the workflow directly inside the report.

Data Import and Cleaning

```
source("R/utils.R")
required_packages(c("readxl"))

raw_dir <- project_path("Raw Data")
processed_dir <- project_path("data", "processed")

read_weather_file <- function(path) {
  canonical_names <- c(
    "longitude_x", "latitude_y", "station_name", "climate_id", "date_time",
    "year", "month", "day", "data_quality", "max_temp_c", "max_temp_flag",
    "min_temp_c", "min_temp_flag", "mean_temp_c", "mean_temp_flag",
    "heat_deg_days_c", "heat_deg_days_flag", "cool_deg_days_c", "cool_deg_days_flag",
    "total_rain_mm", "total_rain_flag", "total_snow_cm", "total_snow_flag",
    "total_precip_mm", "total_precip_flag", "snow_on_grnd_cm", "snow_on_grnd_flag",
    "dir_of_max_gust_10s_deg", "dir_of_max_gust_flag", "spd_of_max_gust_km_h",
    "spd_of_max_gust_flag"
  )

  ext <- tools::file_ext(path)
  if (tolower(ext) == "xlsx") {
    df <- readxl::read_excel(path)
  } else if (tolower(ext) == "csv") {
    df <- read.csv(path, check.names = FALSE)
  }

  if (ncol(df) == length(canonical_names)) {
    names(df) <- canonical_names
  } else {
    names(df) <- snake_case_names(names(df))
  }
  df$source_file <- basename(path)
  df
}
```

Recursive Fixed-Fit ARIMA Helper

```
recursive_fixed_fit_arima_forecast <- function(
  data, target_col, feature_cols, train_end_date,
  order_vec, model_label, eval_dates,
  train_required_cols = NULL, test_required_cols = NULL
) {
  working_data <- data[order(data$date), c("date", target_col, feature_cols), drop = FALSE]

  history_data <- working_data[
    working_data$date <= train_end_date &
      complete.cases(working_data[, c(target_col, feature_cols), drop = FALSE]),
    ,
    drop = FALSE
  ]

  base_fit <- forecast::Arima(
    history_data[[target_col]],
    order = order_vec,
    xreg = as.matrix(history_data[, feature_cols, drop = FALSE]),
    include.mean = TRUE
  )

  for (current_date in eval_dates) {
    test_data <- working_data[working_data$date == current_date, , drop = FALSE]
    model_state <- forecast::Arima(
      history_data[[target_col]],
      model = base_fit,
      xreg = as.matrix(history_data[, feature_cols, drop = FALSE])
    )
    prediction <- forecast::forecast(
```

```
      model_state,
      xreg = as.matrix(test_data[, feature_cols, drop = FALSE]),
      h = 1
    )$mean[1]
  }
}
```

Shared Candidate Screening Setup

```
make_candidate_orders <- function(auto_order, max_p = 3, max_q = 3) {
  p0 <- auto_order[1]
  d0 <- auto_order[2]
  q0 <- auto_order[3]

  candidates <- rbind(
    c(p0, d0, q0),
    c(max(0, p0 - 1), d0, q0),
    c(min(max_p, p0 + 1), d0, q0),
    c(p0, d0, max(0, q0 - 1)),
    c(p0, d0, min(max_q, q0 + 1))
  )

  candidates <- unique(as.data.frame(candidates))
  names(candidates) <- c("p", "d", "q")
  candidates
}

select_finalists <- function(candidate_df, target_values, xreg_matrix = NULL, n_finalists = 1) {
  screening_rows <- do.call(
    rbind,
    lapply(
      seq_len(nrow(candidate_df)),
      function(i) score_candidate_on_training(
        order_vec = as.integer(candidate_df[i, c("p", "d", "q")]),
        target_values = target_values,
        xreg_matrix = xreg_matrix
      )
    )
  )

  screening_rows <- screening_rows[order(screening_rows$screening_aic), ]
  finalists <- head(screening_rows, n_finalists)
  list(screening_rows = screening_rows, finalists = finalists)
}
```

C Code and Data Notes

The submitted archive includes the cleaned datasets in `data/processed/` as csv files, along with the `R` and `Rmd` files used in the analysis. The main script order is: data import and cleaning, feature engineering, Section 2 exploratory plots and summary statistics, baseline and ETS forecasts, ARIMA candidate screening and finalist forecasts, ARIMAX candidate screening and finalist forecasts, forecast comparison of the best models, table export, and best-model diagnostics. This is also the easiest order to follow if someone wants to rerun the project from scratch.